

Software Requirements Specification (SRS)

Project: Traffic Management System (Simulation)

Course: Software Engineering UE24CS341A

Institution: PES University (PESU)

Version: 1.0

Date: 13-09-2026

Status: Draft

Authors

#	Name	SRN
1	Hariom N Kini	PES2UG24CS672
2	Deepthi S	PES2UG24CS632
3	Ramya	PES2UG24CS653
4	Peddinti Suhas	PES2UG24CS647

Revision History

Version	Date	Author	Change Summary	Approval
1.0	13-09-2026	Hariom N Kini, Deepthi S, Ramya, Peddinti Suhas	Finalized SRS for Traffic Management System (Simulation)	Pending

Approvals

Role	Name	Signature / Email	Date
Project Guide	—	—	13-09-2026

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. System models and diagrams (UML Use-Case Diagrams)
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the requirements for the Traffic Management System (Simulation), a C/C++ console-based application that models vehicle queues at a four-way traffic intersection and controls vehicle movement using traffic signals. The system is intended as a Software Engineering college project and is designed to be implemented and demonstrated within one to two weeks.

1.2 Scope

The system shall simulate traffic at one configurable four-way intersection. It shall maintain a separate queue for each approach/lane, generate or accept vehicles, operate traffic signals, move vehicles when permitted, and display the current traffic state and basic statistics through a console interface.

- One four-way intersection with four approaches: North, South, East and West.
- One incoming queue per approach; each queue follows FIFO ordering, implemented using arrays, linked lists, or the C++ Standard Template Library (STL).
- Unique vehicle IDs and simple vehicle attributes such as lane/approach, stored in a struct or class.
- Automatic traffic-signal cycling using configurable green and yellow durations.
- Manual signal control for demonstration/testing via console menu options.
- Automatic vehicle generation at a configurable rate, plus optional manual vehicle insertion via menu input.
- Text-based representation of vehicles approaching and crossing the intersection (per-lane counts and states printed each tick).
- Start, pause, resume, reset and stop controls via console menu.
- Live console display of signal states, queue lengths and processed vehicles.
- End-of-run summary containing waiting vehicles, processed vehicles and basic performance statistics.
- Optional plain-text or CSV log of final statistics written to a local file.

Out of scope:

- Real traffic lights, sensors, cameras, IoT devices or field hardware.
- Internet/network communication.
- Multiple intersections or city-wide road networks.
- Graphical or windowed rendering (GUI), 2D/3D animation.

- Advanced lane-changing, pedestrian, cyclist or public-transport simulation.
- Machine-learning signal optimization or real-time adaptive control.
- Real-world traffic calibration.
- User accounts, cloud storage or a database server.

1.3 Audience

- Development team
- Project guide and course coordinator
- Evaluators
- QA/testers
- Students using the simulator for demonstration

1.4 Definitions and Acronyms

Term	Meaning
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
Queue	Ordered collection of vehicles waiting on an approach, following FIFO order
FIFO	First In, First Out
Approach	One direction from which vehicles enter the intersection
Phase	A period in which a defined set of approaches has permission to move
Simulation tick	One discrete update of the simulator clock/loop iteration
Throughput	Number of vehicles that successfully pass through the intersection during a run
Struct	A C/C++ user-defined composite data type grouping related fields
Node	A single element of a linked-list-based queue, containing data and a pointer to the next node
STL	Standard Template Library (C++), providing containers such as queue and deque
PRNG / Seed	Pseudo-random generator / value used to reproduce generated traffic

1.5 References / Basis

The structure and core queue/signal requirements are based on the team's original Traffic Management System (Simulation) SRS. The expanded functional detail is adapted from the subsequently produced Python/Tkinter SRS, but all GUI-, JSON-, and CSV-specific requirements have been re-expressed for a C/C++ console-based implementation, and the scope remains a single intersection within a short implementation window.

2. Overall Description

2.1 Product Perspective

The application is a standalone C/C++ console program. Its major logical components are:

- Console Interface / Controller (menu-driven input and output)
- Vehicle Manager
- Queue Manager
- Traffic Signal Manager
- Simulation Engine
- Statistics Manager
- Logging / File Output Manager (optional)

Conceptual flow: User → Console Interface → Simulation Engine → Queue/Vehicle + Signal Managers → Statistics → Console Interface.

2.2 Major Product Functions

- Create or generate vehicles.
- Place vehicles in the correct approach queue.
- Maintain FIFO queues using arrays, linked lists, or STL containers.
- Display queue contents/counts in the console.
- Run automatic traffic-signal phases.
- Allow manual signal changes for testing via menu selection.
- Permit vehicle movement only when the approach has a green signal.
- Prevent movement on red or yellow.
- Print vehicle movement/processing events to the console each tick.
- Pause/resume/reset/stop the simulation via menu commands.
- Track waiting and processed vehicles.
- Calculate average waiting time and throughput.
- Optionally write final statistics to a local text or CSV file.

2.3 User Role and Characteristics

Role	Characteristics / Needs
Operator / Analyst	Starts and controls simulations via console menu, adds traffic, changes settings, views live state and results.
Evaluator / Tester	Uses predefined scenarios and acceptance tests to verify requirements.

2.4 Operating Environment

- Windows 10/11 or Linux desktop/laptop.
- C (C11 or later) or C++ (C++11 or later, C++17 recommended for STL features).
- Standard compiler toolchain: GCC/G++, Clang, or MSVC.
- Standard library only (queue, vector/deque, string, ctime, cstdlib); third-party packages should be avoided unless essential.
- Keyboard input and console/terminal output; no display beyond a standard terminal is required.
- No internet connection required.

2.5 Constraints

- Implementation target is one to two weeks for a four-person student team.
- The simulator models one four-way intersection only.
- Traffic behaviour is intentionally simplified.
- Vehicle movement/processing is represented through console text output rather than graphics or animation.
- Only non-conflicting signal phases defined by the application are allowed.
- All data is local and, unless explicitly logged to a file, held only in memory for the duration of the run.
- The project must remain understandable and demonstrable on ordinary student laptops using a standard compiler.

2.6 Assumptions and Dependencies

- A C/C++ compiler and standard library are available on the target machine.
- The user understands basic traffic-signal concepts.
- The application starts with a valid default configuration (default green/yellow durations and generation rate).
- Traffic generation uses a simple configurable probability/rate rather than real traffic data.
- The four-way intersection uses a predefined safe phase sequence unless the user selects manual control.

3. External Interface Requirements

3.1 User Interface

The system shall provide a menu-driven console interface. On each iteration the interface shall present available commands and print the current simulation state.

- Text-based menu for Start, Pause/Resume, Step (optional), Reset and Stop.
- Menu options for vehicle generation: Add Vehicle (manual) and toggle Auto-Generate ON/OFF.
- Signal mode selection: Automatic / Manual, with a menu option to force a specific phase in Manual mode.
- Console prompts to configure green and yellow durations (with input validation).
- Per-tick console output listing North/South/East/West signal state and queue length.
- Running counters for total waiting and total processed vehicles, printed on request or each tick.
- A results summary printed to the console at the end of a run.
- Clear validation/error messages for invalid console input (e.g., non-numeric or out-of-range values).

3.2 Hardware Interfaces

None beyond a standard computer, keyboard and terminal/console display. The software shall not interface with traffic controllers, sensors or cameras.

3.3 Software Interfaces

Interface	Requirement
Language/Runtime	C (C11+) or C++ (C++11+, C++17 recommended)
Compiler	GCC/G++, Clang, or MSVC
Console I/O	Standard input/output (stdio.h / iostream)
Data structures	Arrays, linked lists, or STL containers (std::queue, std::deque, std::vector)
Optional logging	Plain text or CSV file written using standard file I/O (fstream / stdio.h)
OS	Windows/Linux desktop

3.4 Communications

No network communication is required. All simulation processing occurs locally within a single console process.

4. System Features (Detailed Functional Requirements)

4.1 Vehicle Management

Req ID	Requirement (system shall...)	Priority	Acceptance / Test
TMS-F-001	The system shall allow the operator to add a vehicle to a selected approach via a console menu option.	High	Vehicle appears in the selected queue. TC-VM-01
TMS-F-002	The system shall assign each new vehicle to exactly one approach queue.	High	Vehicle count increases by one in the selected approach. TC-VM-02
TMS-F-003	The system shall assign a unique vehicle ID (e.g., an incrementing integer) to every created vehicle.	Medium	No two active/recorded vehicles share an ID. TC-VM-03
TMS-F-004	The system shall display the number of waiting vehicles for each approach in the console output.	High	Displayed count equals queue size. TC-VM-04
TMS-F-005	The system shall support automatic vehicle generation at a user-configurable simple rate (e.g., probability per tick).	Medium	Vehicles are generated while enabled and stop when disabled. TC-VM-05

4.2 Queue Management

Req ID	Requirement (system shall...)	Priority	Acceptance / Test
TMS-F-006	The system shall maintain a separate FIFO queue for each of the four approaches, implemented using an array-based circular buffer, a singly linked list, or an STL container (std::queue/std::deque).	High	Vehicles remain in arrival order. TC-Q-01
TMS-F-007	The system shall add new vehicles to the rear of the selected queue (enqueue).	High	New vehicle becomes last in queue. TC-Q-02
TMS-F-008	The system shall remove the front vehicle from a queue (dequeue) only when movement is permitted.	High	Front vehicle is removed/processed; later vehicles remain ordered. TC-Q-03
TMS-F-009	The system shall safely handle a dequeue attempt on an empty queue without crashing (e.g., by checking an is-empty condition before accessing memory).	High	Attempted removal from empty queue is detected and handled cleanly (no null-pointer dereference or undefined behavior). TC-Q-04
TMS-F-010	The system shall update queue counts immediately after an enqueue or dequeue operation.	High	Displayed count matches internal queue size. TC-Q-05

4.3 Traffic Signal Management

Req ID	Requirement (system shall...)	Priority	Acceptance / Test
TMS-F-011	The system shall display the current signal state for each approach as Red, Yellow or Green (represented internally by an enum or integer constant).	High	Displayed states match signal-manager state. TC-SIG-01
TMS-F-012	Automatic mode shall cycle through a predefined safe phase sequence.	High	Only configured non-conflicting approaches receive green. TC-SIG-02
TMS-F-013	The system shall support configurable green and yellow phase durations within accepted limits, entered via console input.	Medium	Changed duration is reflected in the next applicable phase. TC-SIG-03
TMS-F-014	The system shall allow manual signal selection via console menu when Manual mode is enabled.	Medium	Selected safe phase is applied and displayed. TC-SIG-04
TMS-F-015	The system shall prevent a vehicle from entering the intersection while its approach is Red or Yellow.	High	Vehicle remains waiting. TC-SIG-05

4.4 Traffic Flow Simulation

Req ID	Requirement (system shall...)	Priority	Acceptance / Test
TMS-F-016	The system shall run the simulation using discrete time ticks controlled by a loop (e.g., a while loop with a sleep/delay call or tick counter).	High	Simulation clock/tick counter advances repeatedly while running. TC-SIM-01
TMS-F-017	The system shall move (dequeue and mark processed) the front vehicle from a green approach on each eligible tick.	High	Vehicle is dequeued and counted as processed. TC-SIM-02
TMS-F-018	The system shall prevent conflicting approach movements from occurring in the same phase.	High	No conflicting green/movement state occurs. TC-SIM-03
TMS-F-019	The system shall provide Start, Pause/Resume, Reset and Stop commands via the console menu.	High	Each control produces the specified state transition. TC-SIM-04
TMS-F-020	Reset shall return queues, counters, signals and simulation time/tick count to the initial state.	High	All state returns to defaults. TC-SIM-05
TMS-F-021	The system shall continue running after normal invalid user input is rejected.	High	Invalid input produces a message without program termination. TC-SIM-06

4.5 Monitoring and Statistics

Req ID	Requirement (system shall...)	Priority	Acceptance / Test
TMS-F-022	The system shall display total vehicles currently waiting.	High	Total equals sum of four queue sizes. TC-MET-01
TMS-F-023	The system shall count vehicles that successfully cross the intersection.	High	Processed counter increments exactly once per completed vehicle. TC-MET-02
TMS-F-024	The system shall calculate average waiting time for vehicles processed during a run, using recorded enqueue and dequeue tick/time values.	Medium	Value matches recomputation from recorded wait times within rounding. TC-MET-03
TMS-F-025	The system shall calculate throughput as processed vehicles per simulated run duration (ticks or seconds).	Medium	Value matches processed/time calculation. TC-MET-04
TMS-F-026	When the simulation stops, the system shall print a summary of waiting, processed and basic performance statistics by approach.	High	Summary values match final simulation state. TC-MET-05

4.6 Logging and Data Management

Req ID	Requirement (system shall...)	Priority	Acceptance / Test
TMS-F-027	The system shall optionally allow the operator to write the current configuration (durations, generation rate) to a local plain-text file.	Low	Written file contains signal and traffic-generation settings. TC-DATA-01
TMS-F-028	The system shall optionally allow a previously written configuration file to be read back in at startup or on request.	Low	Loaded values appear in the console configuration display. TC-DATA-02
TMS-F-029	The system shall optionally export final statistics as a CSV file using standard file I/O.	Low	CSV contains headers and final displayed metrics. TC-DATA-03

5. Non-Functional Requirements

Req ID	Requirement (system shall...)	Priority	Acceptance / Test
TMS-NF-001	Performance: On a normal student laptop, the console loop shall process a tick and update all queues/signals without noticeable delay for the supported scenario size (up to 100 active vehicles).	High	30-minute stress run (or equivalent tick count) with 100 active vehicles completes without crash or persistent freezing. TC-PERF-01
TMS-NF-002	Reliability: Valid simulation operations shall produce consistent, deterministic state transitions given the same seed/inputs.	High	Repeated deterministic test scenario produces expected counts. TC-REL-01
TMS-NF-003	Usability: Console menu options and printed status shall be understandable without external documentation.	Medium	A tester can start, pause, reset and view queue/signal state using only on-screen menu text. TC-UX-01
TMS-NF-004	Maintainability: Vehicle/queue, signal, simulation and statistics logic shall be separated into distinct source files, structs/classes, and functions.	Medium	Each logical component (e.g., queue.c/queue.h, signal.c/signal.h) can be modified without rewriting the entire application.
TMS-NF-005	Portability: The application shall compile and run on supported Windows/Linux machines with a standard C/C++ compiler, avoiding platform-specific APIs where possible.	Medium	Application compiles and completes a smoke test on two supported environments when available. TC-PORT-01
TMS-NF-006	Safety: The simulation shall never intentionally allow conflicting approaches to receive green in automatic mode.	High	Automated signal-cycle test finds no conflicting green combination. TC-SAFE-01
TMS-NF-007	Memory safety: The application shall not leak memory or access invalid pointers during normal operation (relevant to manual memory management if linked lists / dynamic allocation are used).	High	A memory-checking tool (e.g., Valgrind) reports no leaks or invalid accesses during a standard test run. TC-MEM-01

6. Quality attributes & Acceptance tests

6.1 Security Objectives

- Protect the application from invalid user input (non-numeric, out-of-range, or malformed console input).
- Preserve internal vehicle, queue and signal-state integrity.
- Prevent invalid queue operations (e.g., dequeue from an empty queue) and unsafe signal combinations.
- Prevent memory-safety issues such as buffer overflows, out-of-bounds array access, and dangling pointers when using manual memory management.
- Handle malformed local configuration/log files gracefully, if file I/O is implemented.

6.2 Representative Acceptance Tests

Req ID	Requirement (system shall...)	Priority	Acceptance / Test
TMS-SR-001	The system shall validate numeric settings such as phase durations and generation rates before applying them (reject negative, zero-where-invalid, or non-numeric console input).	High	Invalid/negative/non-numeric values are rejected with a clear message. TC-SEC-01
TMS-SR-002	The system shall prevent dequeue operations on empty queues by checking queue state before accessing array indices or pointers.	High	No out-of-bounds access or null-pointer dereference occurs. TC-SEC-02
TMS-SR-003	The system shall validate any loaded configuration file before applying it, rejecting malformed or incomplete data.	Medium	Malformed or incomplete configuration is rejected with an error. TC-SEC-03
TMS-SR-004	Unexpected user input shall not cause abnormal program termination (e.g., via unhandled exceptions in C++ or unchecked scanf/cin failures in C/C++).	High	Application remains usable after invalid input. TC-SEC-04
TMS-SR-005	If dynamic memory allocation is used (e.g., linked-list nodes), the system shall free all allocated memory on program exit and on queue/vehicle removal.	High	No memory leaks detected by a memory-checking tool across a full run. TC-SEC-05

7. UML Use-Case Diagram

7.1 Key Quality Attributes

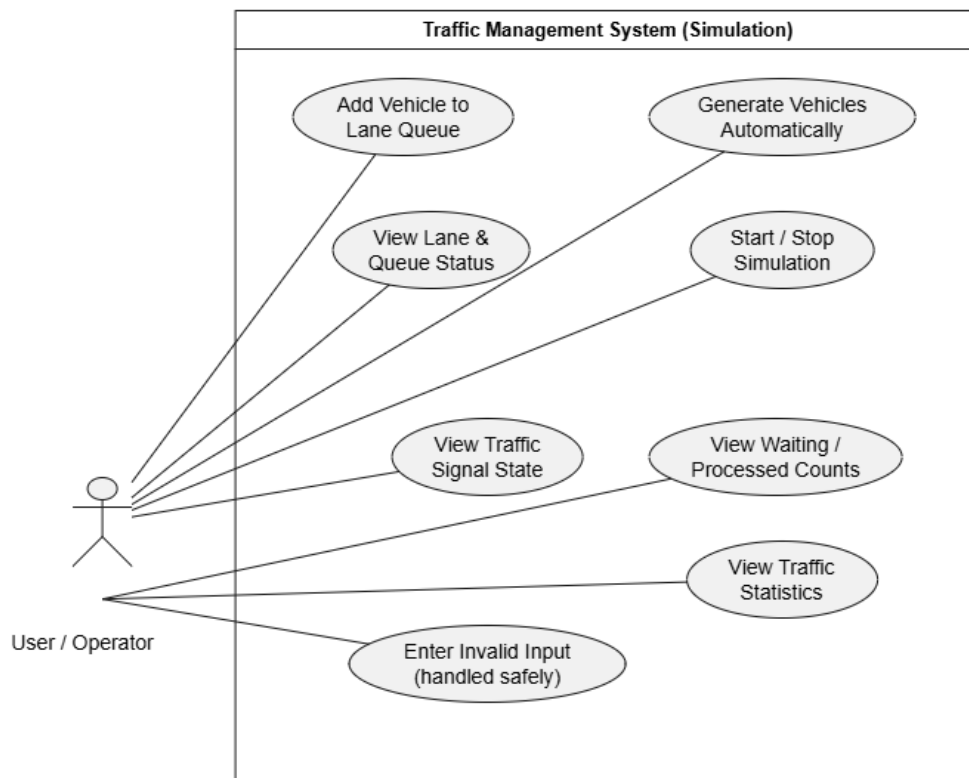
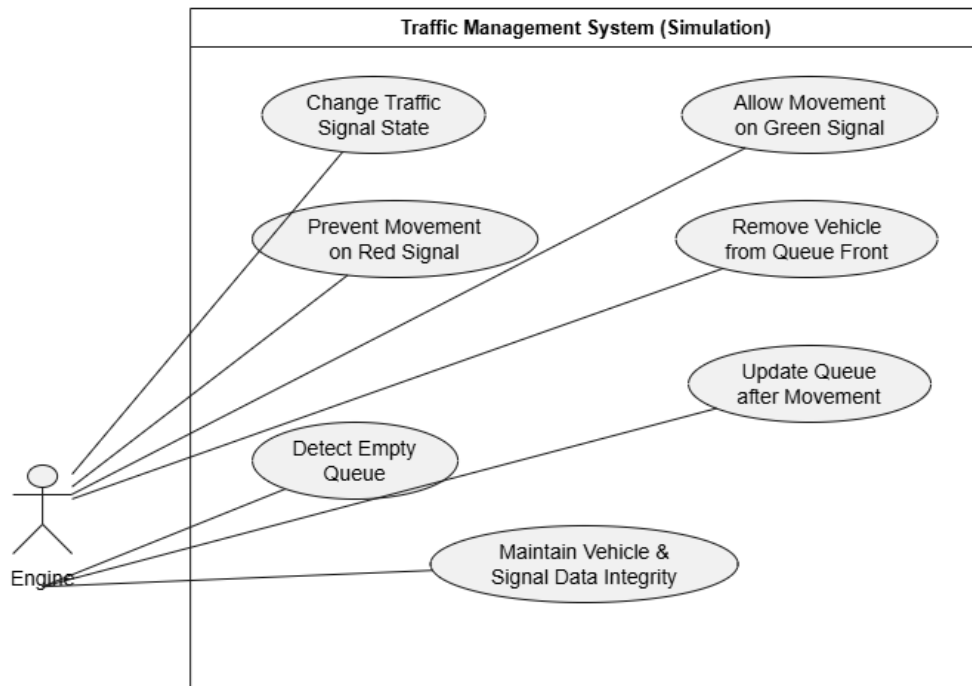
- Correctness — queue and signal rules are followed.
- Reliability — normal operations do not crash the application.
- Usability — the console menu workflow is clear.
- Performance — supported traffic load remains responsive within the tick loop.
- Maintainability — logic is divided into manageable C/C++ source files, structs, and functions.
- Portability — application can compile and run on supported desktop environments with a standard compiler.
- Memory safety — no leaks, buffer overruns, or invalid pointer accesses.
- Testability — queue, signal and statistics logic can be tested independently of console I/O (e.g., via functions callable from a test harness).

7.2 Acceptance / Exit Criteria

- All High-priority functional requirements are implemented.
- All High-priority acceptance tests pass.
- All NFRs are checked and no Critical/High defect remains.
- The application can start a simulation, process vehicles, pause, reset and stop via the console menu.
- The final summary shows correct queue and processed counts.
- No memory leaks or invalid memory accesses are reported by a memory-checking tool.
- The demonstration can be completed without internet or specialized hardware, using only a terminal.

7.3 Acceptance Test Suite

Test ID	Test	Expected Result
TC-VM-01	Add vehicle to each approach	Correct queue count increases.
TC-Q-01	Add 5 vehicles to one approach	Vehicles leave in FIFO order.
TC-Q-04	Attempt movement from empty queue	No crash; queue remains empty; state handled cleanly.
TC-SIG-02	Run automatic signal cycle	Only safe configured phase is green.
TC-SIG-05	Vehicle faces red/yellow	Vehicle does not cross.
TC-SIM-02	Run with queued vehicles on green	Front vehicles move and are processed.
TC-SIM-05	Reset during run	Queues, counters and tick count return to defaults.
TC-MET-03	Process vehicles with known waits	Average wait matches expected value.
TC-SEC-01	Enter invalid duration/rate	Input is rejected safely.
TC-MEM-01	Run full simulation under Valgrind (or equivalent)	No memory leaks or invalid accesses reported.
TC-DATA-01	Save configuration (if implemented)	Text/CSV file is created with settings.
TC-PERF-01	Stress with supported load	No crash/persistent freezing.



8. Requirements Traceability Matrix (RTM)

Status is N (Not started), P (Partially verified), or A (Accepted). The team should update the status after each test.

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
TMS-F-001–005	Vehicle management	4.1 / DS-Vehicle-01	vehicle.c / ui.c	TC-VM-01...05	N	
TMS-F-006–010	Queue management	4.2 / DS-Queue-01	queue.c	TC-Q-01...05	N	
TMS-F-011–015	Signal management	4.3 / DS-Signal-01	signal.c	TC-SIG-01...05	N	
TMS-F-016–021	Traffic flow simulation	4.4 / DS-Sim-01	simulation.c / ui.c	TC-SIM-01...06	N	
TMS-F-022–026	Monitoring & statistics	4.5 / DS-Stats-01	statistics.c / ui.c	TC-MET-01...05	N	
TMS-F-027–029	Optional data management	4.6 / DS-Data-01	fileio.c	TC-DATA-01...03	N	Optional/Low priority
TMS-NF-001	Performance	5 / DS-Perf-01	simulation.c / ui.c	TC-PERF-01	N	

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
TMS-NF-002	Reliability	5 / DS-Rel-01	All core modules	TC-REL-01	N	
TMS-NF-003	Usability	5 / DS-UX-01	ui.c	TC-UX-01	N	
TMS-NF-004	Maintainability	5 / DS-Maint-01	All modules	TC-MAINT-01	N	
TMS-NF-005	Portability	5 / DS-Port-01	Build scripts	TC-PORT-01	N	
TMS-NF-006	Safety (no signal conflicts)	5 / DS-Safe-01	signal.c	TC-SAFE-01	N	
TMS-NF-007	Memory safety	5 / DS-Mem-01	queue.c / vehicle.c	TC-MEM-01	N	
TMS-SR-001–005	Security requirements	5.1.2 / DS-Sec-01	ui.c / fileio.c / queue.c	TC-SEC-01...05	N	